COMPLETE PROJECT AUDIT

Cadence

AI-powered academic planner for university workload forecasting

Prepared for: Cadence project review
Audit date: 2026-06-17
Repository: /Users/dakshrathod/Desktop/cadence
Review stance: Read-only technical, product, security, and readiness audit
Overall readiness: 22 / 100

Executive Finding

Cadence is a credible MVP prototype with a working rule-based planner, FastAPI endpoint, and polished single-page Next.js interface. It is not production-ready: authentication, Supabase/PostgreSQL persistence, RLS, integrations, OpenAI API usage, notification infrastructure, and production backend hardening are absent.

1. Current Project Status

Feature	Status	Completion	Notes
Authentication	Not Started	0%	No Supabase client, auth middleware, login UI, sessions, protected routes, or user model.
Student Dashboard	In Progress	55%	Single-page dashboard exists, but uses local/browser state.
Assignment & Deadline Management	Partially Implemented	45%	Manual add/edit/delete exists; no backend CRUD, course model, ownership, or sync.
Calendar View	Not Started	0%	Sidebar item exists but is disabled.
Google Classroom Integration	Not Started	0%	No OAuth, Classroom API client, sync jobs, or token storage.
Moodle Integration	Not Started	0%	No Moodle API client, import flow, or credentials.
AI Workload Forecasting	Partially Implemented	35%	Rule-based deterministic planner exists; no OpenAI API usage.

AI Study Plan Generation	Partially Implemented	40%	Backend returns study sessions; not personalized, editable, persistent, or AI-generated.
Analytics & Productivity Insights	Partially Implemented	25%	Basic derived metrics exist; no history, trends, productivity tracking, or cohorts.
Notifications & Reminders	Not Started	0%	Bell icon only; no jobs, email, push, or preferences.
Backend API	In Progress	25%	FastAPI exposes /health and /plan only.
Database / Supabase / PostgreSQL	Not Started	0%	No migrations, tables, RLS, indexes, or Supabase config.
Deployment	Partially Implemented	35%	Frontend builds and is documented as hosted; backend is local-only.
Testing	Partially Implemented	25%	Planner unit tests pass; no frontend, API integration, E2E, auth, DB, or security tests.

2. Working Features

- Backend planner tests pass: 5 planner tests succeed.
- Frontend production build succeeds with Next.js.
- FastAPI /health returns service status.
- FastAPI /plan returns workload weeks, collisions, study sessions, summary, and suggestion.
- Manual deadline entry, edit, and delete work client-side.
- Planner state persists in localStorage for the current browser.
- Sample DAU data can be loaded and planned.
- Dashboard metrics, workload chart, collision panel, study plan, and analytics render from planner output.

3. Missing Features

Feature	Expected User Flow	Dependencies
Auth	Signup, login, session, logout, protected dashboard.	Supabase Auth, middleware, session helpers, UI.
Persistent academic data	Courses, assignments, plans, settings survive refresh and device changes.	PostgreSQL schema, RLS, CRUD APIs.

Calendar view	Month/week/day calendar for deadlines and study blocks.	Stored deadlines/plans, calendar component.
Google Classroom import	Connect Google, review coursework, confirm sync.	Google OAuth, Classroom API, token storage.
Moodle import	Connect Moodle, import deadlines, map courses.	Moodle API/token flow, institution config.
OpenAI explanations	Explain plan tradeoffs and schedule changes.	OpenAI API, prompts, cost and rate controls.
Reminders	Email/push/in-app alerts before deadlines.	Notification schema, cron/queue, email/push provider.
Analytics history	Completion tracking and productivity trends.	Session status data and historical tables.
Settings/profile	Capacity, timezone, peak hours, study preferences.	Profile table and settings UI.
Production backend	Hosted, observable, secured API.	Deployment config, envs, CORS, logs.

4. Technical Debt

Rank	Issue
Critical	No authentication or authorization. Any persisted API would be unsafe without user isolation.
Critical	No database schema, migrations, RLS, or persistence despite Supabase/PostgreSQL being core stack.
High	Planner API is unauthenticated and lacks rate limiting, request size limits, abuse protection, and production CORS.
High	Frontend directly calls NEXT_PUBLIC_API_URL with no typed server API boundary, timeout, retry, or runtime validation.
High	npm run lint is broken/interactively blocked because next lint prompts for setup.
Medium	Planner algorithm ignores availability, due time, weekends, existing commitments, and personalized preferences.
Medium	Several UI controls are presentational only: Save Plan, bell, logout, New Study Session, Apply Suggestion, Review/Dismiss.
Medium	localStorage stores academic data unencrypted and unsynced.
Medium	API failures collapse into one generic frontend error.
Low	CORS only allows localhost; no deployed backend origin

	configured.

5. Database Review

Current database status: no Supabase project files, SQL migrations, schema definitions, seed files, generated types, RLS policies, indexes, or database access code exist.

Table	Purpose	Key Relationships / Notes
profiles	Student profile/settings	id references auth.users; timezone, weekly capacity, peak hours.
courses	Student courses	user_id, course code, name, term.
assignments	Deadlines/tasks	user_id, course_id, title, due_at, estimated_hours, difficulty, priority, source.
study_plans	Generated plans	user_id, input snapshot, generated_by, summary.
study_sessions	Scheduled work blocks	plan_id, assignment_id, starts_at, ends_at, status.
integrations	Connected providers	provider, user_id, encrypted tokens/ref IDs.
notifications	Reminder queue/history	assignment/session target, scheduled_at, sent_at, channel.

Required improvements: enable RLS on every user-owned table; add indexes on user_id, due_at, course_id, plan_id, and scheduled_at; generate Supabase TypeScript types; store provider tokens securely rather than plaintext in application tables.

6. API Review

Existing APIs

Method	Path	Status	Notes
GET	/health	Working	Simple health check.
POST	/plan	Working MVP	Stateless planner; no auth, persistence, user context, OpenAI, or idempotency.

Recommended APIs

API	Purpose
GET/POST /api/courses	Course CRUD.
GET/POST/PATCH/DELETE /api/assignments	Deadline CRUD.
POST /api/plans/generate	Authenticated generation and persisted plan output.
GET /api/plans/latest	Load latest plan.
PATCH /api/study-sessions/:id	Reschedule or complete sessions.
POST /api/integrations/google/connect	Google OAuth start/callback.
POST /api/integrations/moodle/sync	Moodle sync.
GET/POST /api/notifications	Reminder preferences and queue.
POST /api/ai/explain-plan	OpenAI-generated explanation with guardrails.

7. UI/UX Review

The UI is polished for an MVP: clear dashboard, responsive mobile navigation, good visual hierarchy, and useful first-run empty states. The main product risk is trust: multiple controls look live but do not perform product actions.

- Disable, hide, or implement non-functional actions until they are real.
- Add onboarding for first-time users and explain why connecting data sources matters.
- Make Calendar and AI Assistant either functional or visibly marked as upcoming.
- Add accessible labels for icon-only buttons and improve keyboard/focus behavior.
- Add form validation messages and timezone-aware due dates.
- Replace static Study Planner values such as 25h, CS401 Project, and Morning with real data.

8. Priority Roadmap

PHASE 1 - Must Have

Task	Complexity	Time	Dependencies
Add Supabase Auth and protected app shell	High	2-4 days	Supabase project/envs.
Create database schema, migrations, RLS	High	2-3 days	Auth model.
Replace localStorage with DB-backed CRUD	High	3-5 days	Schema, API client.
Deploy backend or move	Medium	1-2 days	Env/deploy choice.

planner into Next API route			
Fix lint/test/build pipeline	Medium	0.5-1 day	ESLint config.
Add production env examples and docs	Low	0.5 day	Final env list.

PHASE 2 - Important

Task	Complexity	Time	Dependencies
Persist generated plans and study sessions	Medium	2-3 days	DB CRUD.
Build Calendar view	Medium	2-4 days	Persisted sessions.
Add OpenAI explanation endpoint	Medium	1-2 days	OpenAI key, prompt/eval.
Add reminder system	High	3-5 days	Cron/queue/email.
Add frontend/API integration tests	Medium	2-3 days	Stable APIs.
Add observability/logging	Medium	1-2 days	Deployment.

PHASE 3 - Nice to Have

Task	Complexity	Time	Dependencies
Google Classroom import	High	5-8 days	OAuth, token storage.
Moodle import	High	5-10 days	Institution-specific API details.
Productivity insights and adherence analytics	Medium	3-5 days	Session completion data.
Collaborative/group plans	High	5-10 days	Sharing/permissions.
Calendar export	Medium	2-3 days	Stable event model.

9. Codex Execution Plan

Task	Objective	Files to Modify	Acceptance Criteria	Time
Task 1	Fix lint pipeline and add CI-quality scripts.	frontend/ package.json, ESLint config files.	npm run lint, npm run build, and backend tests run	1 hour

			non-interactively.	
Task 2	Add Supabase env/config and generated client structure.	frontend/src/lib/supabase/*, .env.example, docs.	App initializes browser/server Supabase clients without secrets in client bundle.	2 hours
Task 3	Add database migrations with RLS.	supabase/migrations/*.	Profiles, courses, assignments, plans, sessions, integrations, notifications tables exist with user-scoped RLS.	1 day
Task 4	Implement authentication UI and protected routes.	App routes, middleware, auth components.	Unauthenticated users cannot access planner; login/logout works.	1-2 days
Task 5	Replace localStorage deadlines with Supabase CRUD.	PlannerApp, deadline components, data access layer.	Deadlines persist per user across refresh/devices.	2 days
Task 6	Persist generated plans/sessions.	Planner API layer, DB actions, study planner UI.	Latest generated plan reloads from DB and sessions can be marked complete.	2 days
Task 7	Productionize planner API.	Backend or Next API route, deployment config, CORS/env docs.	Deployed frontend calls deployed planner securely.	1-2 days
Task 8	Build calendar view.	Calendar route/component, session queries.	Deadlines and study blocks appear in month/week view.	3 days
Task 9	Add OpenAI-powered explanation endpoint.	AI service, prompt templates, API route, UI panel.	Suggestions are generated safely from user-owned plan data with fallback behavior.	2 days
Task 10	Add reminders.	Notification schema/actions, cron job, email/push adapter, settings UI.	Users can configure and receive scheduled reminders.	4 days
Task 11	Add Google Classroom integration.	OAuth routes, integration tables, sync service, import review UI.	User can import coursework deadlines into assignments.	1 week
Task 12	Add Moodle	Moodle connector,	User can import	1 week

	integration.	credential flow, sync service, import review UI.	Moodle deadlines reliably.	

10. Production Readiness Score

Area	Score
Frontend	45 / 100
Backend	25 / 100
Database	0 / 100
Security	8 / 100
AI Features	15 / 100
Integrations	0 / 100
UX	45 / 100
Scalability	15 / 100
Overall	22 / 100

Readiness Summary

Cadence validates the planner concept, but the production foundation is missing. The next milestone should be a secure, authenticated, persistent single-user planner before external integrations or advanced AI work.

Final Checklist Ordered by Priority

Priority 1-8	Priority 9-15
1. Fix lint/CI scripts.	9. Add calendar view.
2. Add .env.example and production environment documentation.	10. Add OpenAI explanation endpoint with fallbacks.
3. Create Supabase schema, migrations, indexes, and RLS.	11. Add reminder scheduling and delivery.
4. Implement Supabase Auth and protected routes.	12. Add frontend/API/E2E tests.
5. Replace localStorage with authenticated database CRUD.	13. Add observability, rate limits, and error monitoring.
6. Persist generated plans and study sessions.	14. Build Google Classroom import.
7. Secure and deploy the planner API.	15. Build Moodle import.

8. Remove or implement non-functional UI controls.	